point. By default, this generator expects a two-part array, where the first part is the x value and the second part is the y value. We can't use that, because our x value is based on the day of the activity and our y value is based on the amount of activity.

The `.x()` accessor function of the line generator needs to point at the scaled day value, while the `.y()` accessor function needs to point to the scaled value of the appropriate activity. The line function itself takes the entire dataset that we loaded from tweet-data, and returns the SVG drawing code necessary for a line between the points in that dataset. To generate three lines, we use the dataset three times, with a slightly different generator for each. We not only need to write the generator function and define how it accesses the data it uses to draw the line, but we also need to append a `<path>` to our canvas and set its `d` attribute to equal the generator function we defined.

Listing 4.9 New line generator code inside the callback function

```
var tweetLine = d3.svg.line()
    .x(function(d) {
        return xScale(d.day);
    })
    .y(function(d) {
    return yScale(d.tweets);
    });

d3.select("svg")
    .append("path")
    .attr("d", tweetLine(data))
    .attr("fill", "none")
    .attr("stroke", "darkred")
    .attr("stroke-width", 2);
```

Defines an accessor for data like ours; in this case we take the day attribute and pass it to xScale first

This accessor does the same for the number of tweets.

The appended path is drawn according to the generator with the loaded tweetdata passed to it.

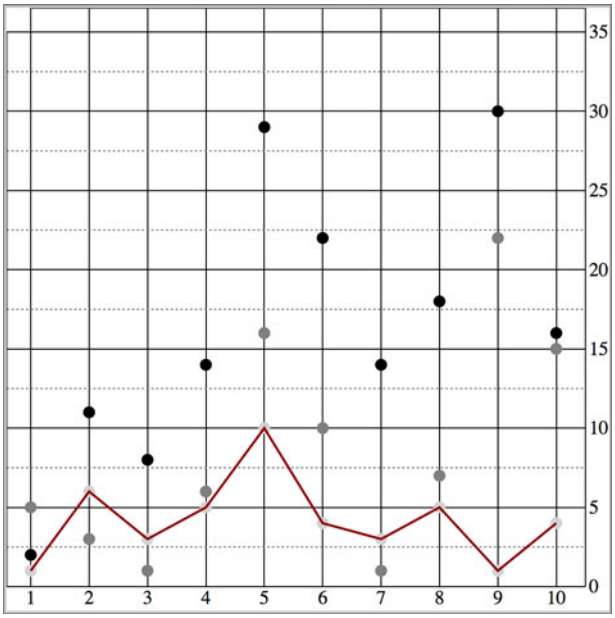


Figure 4.18 The line generator takes the entire dataset and draws a line where the x,y position of every point on the canvas is based on its accessor. In this case, each point on the line corresponds to the day, and tweets are scaled to fit the x and y scales we created to display the data on the canvas.